# Documentation

Message Passing Implementation in xv6

Name : Korrapati Sai Sreeram
Ad.No : 24JE0641

## 1. Message Passing — Idea

Message passing is a mechanism of Inter-Process Communication (IPC) where processes communicate by sending and receiving messages through the kernel. In this implementation, two system calls — **msgsend** and **msgrecv** — are added to xv6 to allow processes to exchange messages safely without sharing memory.

## 2. Data Structure — Message Box (proc.h)

A message structure is added inside each process (**struct proc**) in **kernel/proc.h** to store multiple messages in a queue-like structure. Each process can hold up to **8 messages**, each up to **256 bytes** in size.

### Fields of the message struct:

- **sender_pid** — stores the PID of the process that sent the message
- **used** — flag indicating if this slot is occupied (1 = full, 0 = empty)
- **data[256]** — buffer holding the actual message content
- **size** — length of the stored message

```
  struct message {
    int sender_pid;
    int used;
    char data[256];
    int size;
  } msgbox[8];
};
```

*Figure 1: struct message added inside struct proc in kernel/proc.h*

## 3. System Call: msgsend

The **sys_msgsend** function is implemented in **kernel/sysproc.c**. It allows one process to send a message to another process identified by its PID.

### Explanation:

- Retrieves the target process ID using **argint()**
- Retrieves the message from user space using **argstr()**
- Ensures proper null termination of the message (buf[255] = 0)
- Calculates actual message size using **strlen()**
- Traverses the process table **proc[]** to find the target process by PID
- Searches for an empty slot in the process message box (**msgbox**)
- Stores sender PID, message content, and message size in the slot
- Null terminates the stored message to prevent garbage output
- Returns **0** on success, **-1** if mailbox is full or process not found

```c
uint64
sys_msgsend(void)
{
  int pid, size;
  char buf[256];
  struct proc *p;

  argint(0, &pid);
  argint(2, &size);
  if(argstr(1, buf, sizeof(buf)) < 0)
    return -1;

  // Fix 1: force null termination
  buf[255] = 0;
  // Fix 2: use actual string length
  size = strlen(buf) + 1;

  for(p = proc; p < &proc[NPROC]; p++){
    if(p->pid == pid){
      for(int i = 0; i < 8; i++){
        if(p->msgbox[i].used == 0){
          p->msgbox[i].used = 1;
          p->msgbox[i].sender_pid = myproc()->pid;
          p->msgbox[i].size = size;
          memmove(p->msgbox[i].data, buf, size);
          // Fix 3: null terminate stored message
          p->msgbox[i].data[size-1] = 0;
          return 0;
        }
      }
      return -1;
    }
  }
  return -1;
}
```

*Figure 2: sys_msgsend() implementation in kernel/sysproc.c*

## 4. System Call: msgrecv

The **sys_msgrecv** function is implemented in **kernel/sysproc.c**. It allows a process to receive a message from its message queue.

### Explanation:

- Retrieves the user buffer address using **argaddr()**
- Retrieves buffer size using **argint()**
- Accesses the current process using **myproc()**
- Traverses the message queue (**msgbox[]**) to find a used slot
- Finds the first available message where **used == 1**
- Null terminates the message data before copying
- Copies the message to user space using **copyout()**
- Clears the message slot by setting **used = 0** after reading
- Returns the **sender PID** on success, **-1** if no message is available

```c
uint64
sys_msgrecv(void)
{
  int size;
  uint64 buf;
  struct proc *p = myproc();

  argaddr(0, &buf);
  argint(1, &size);

  for(int i = 0; i < 8; i++){
    if(p->msgbox[i].used == 1){
      int sz = p->msgbox[i].size < size ? p->msgbox[i].size : size;
      // Fix: always null terminate output
      p->msgbox[i].data[sz-1] = 0;
      if(copyout(p->pagetable, buf, p->msgbox[i].data, sz) < 0)
        return -1;
      p->msgbox[i].used = 0;
      return p->msgbox[i].sender_pid;
    }
  }
  return -1;
}
```

*Figure 3: sys_msgrecv() implementation in kernel/sysproc.c*

## 5. Registering the System Calls

### 5a. System Call Numbers (kernel/syscall.h)

Unique numeric identifiers are assigned to the new system calls in **kernel/syscall.h**. These numbers are used by the kernel dispatcher to route system call requests.

```c
#define SYS_msgsend 22
#define SYS_msgrecv 23
```

*Figure 4: SYS_msgsend = 22 and SYS_msgrecv = 23 defined in kernel/syscall.h*

### 5b. Syscall Dispatch Table (kernel/syscall.c)

The system calls are registered in the kernel's dispatch table in **kernel/syscall.c**. This connects the syscall numbers to their corresponding kernel handler functions.

```c
[SYS_msgsend]  sys_msgsend,
[SYS_msgrecv]  sys_msgrecv,
```

*Figure 5: syscall table entries in kernel/syscall.c*

### 5c. User-Level Declarations (user/user.h)

Function prototypes are defined in **user/user.h** so that user programs can call the new system calls just like any standard library function.

```
int msgsend(int, char*, int);
int msgrecv(char*, int);
```

*Figure 6: msgsend and msgrecv declared in user/user.h*

### 5d. Assembly Stubs (user/usys.pl)

Entries are added in **user/usys.pl** to auto-generate the assembly stubs that trigger the system call trap into the kernel.

```
entry("msgsend");
entry("msgrecv");
```

*Figure 7: syscall entries in user/usys.pl*

## 6. User Program — msgtest.c

A user-level test program **msgtest.c** is created to demonstrate message passing between a parent and child process. The program forks a child, the parent sends a message using **msgsend**, and the child continuously polls using **msgrecv** until it receives the message. The parent uses **wait()** to ensure the child finishes before printing the final output, preventing interleaved console output.

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main() {
  int pid;
  char msg[] = "Hello from parent!";
  char buf[256];

  printf("=== Message Passing Test ===\n");

  pid = fork();

  if(pid < 0){
    printf("Fork failed\n");
    exit(1);
  }

  if(pid == 0) {
    // Child process

    int sender;
    while((sender = msgrecv(buf, sizeof(buf))) < 0);

    printf("Child (PID %d): got message from PID %d: %s\n",
           getpid(), sender, buf);

    exit(0);

  } else {
    // Parent process

    int r = msgsend(pid, msg, strlen(msg) + 1);

    wait(0);  // wait FIRST → ensures child prints first

    if(r < 0){
      printf("Parent (PID %d): msgsend failed\n", getpid());
    } else {
      printf("Parent (PID %d): sent message to child (PID %d)\n",
             getpid(), pid);
    }

    printf("=== Test Complete ===\n");
    exit(0);
  }
}
```

*Figure 8: msgtest.c — user program demonstrating message passing*

## 7. Output

The output below shows successful inter-process communication. The child process receives the message **"Hello from parent!"** from the parent and prints it along with the sender's PID. The parent then confirms it sent the message successfully.

```
$ msgtest
=== Message Passing Test ===
Child (PID 5): got message from PID 4: Hello from parent!
Parent (PID 4): sent message to child (PID 5)
=== Test Complete ===
```

*Figure 9: xv6 terminal output showing successful message passing*

## 8. How It Works

- Parent process sends a message using **msgsend(pid, msg, size)**
- The kernel locates the target process by PID and deposits the message in its **msgbox[]**
- Child process continuously polls using **msgrecv(buf, size)** until a message arrives
- Once received, the message is copied from kernel space to user space via **copyout()**
- The child prints the message along with the sender's PID
- Synchronization is ensured using **wait()** to avoid output interleaving

## 9. Conclusion

Message passing was successfully implemented in xv6 using two custom system calls — **msgsend** and **msgrecv**. This approach enables safe communication between processes through the kernel without requiring shared memory. Each process maintains its own message queue (**msgbox**) embedded in the process structure, allowing up to 8 pending messages. The implementation demonstrates key operating system concepts including system call design, process management, kernel-user space data transfer, and inter-process communication.